

Protractor for Angular E2E Testing

- Protractor is a very popular solution positioned as a framework for testing Angular applications.
 - Protractor has built-in means to work with Angular elements. This simplifies code significantly and allows you to write tests easily and fast.
 - Protractor tests against real browsers, which is an important advantage.
 - Alternative to protractor could be Cypress E2E or nightwatch.js .
-
- Protractor allows us to automate actions within the browser to simulate an actual user interacting with the page.
 - We can trigger clicks on buttons and enter text into form fields in an automated process that runs the same each time.

Installation and configuration:

- Protractor is a tool developed to run end to end tests, by the angular team. It is based on **selenium** and can run scenarios in various browsers.
- Selenium is a set of tools that allows for the automation of web browsers, generally for web application testing purposes.
- The default project template created by Protractor depends on two files to run the tests:
 - the spec files that reside inside the **e2e** directory and
 - the configuration file (**protractor.conf.js**)

If you are ok with running the test on Chrome web browser, you can leave the configuration file **protractor.conf.js** as is and need not Install Protractor for with Selenium Standalone Server. Chrome makes use of the Selenium - WebDriver

In angular-cli you have already preconfigured task for it:

ng e2e

Protractor Basics

- The spec file, ending with **.e2e-spec.ts**, will host the actual tests for our application.
- There are two things you need to consider while writing Protractor tests: Jasmine Syntax and Protractor API (<https://www.protractortest.org/#/api>)

Jasmine Syntax:

- Our tests will be organized inside the spec file using Jasmine's syntax.
- Jasmine has a great syntax for writing tests, and it works just as well with Protractor.
- As for unit test, `describe()`, `beforeEach()`, and `it()` are global Jasmine functions.
- The good news is that the switching costs between testing frameworks is relatively low with differences in syntax as small as Jasmine's `toEqual()` and Mocha's `to.equal()`.

Protractor Basics

The prominent components exported by Protractor API are listed below:

(1) `browser()`. Example:

```
beforeEach(() => {browser.get('/');});
```

(2) `element()`. Example:

```
it('should display the name of the application',() => {  
  expect(element(by.css('.pastebin')).getText()).toContain('Pastebin  
Application');  
});
```

(3) `element.all()`. This is used to look for an array of elements that match some chain of conditions.

(4) Locators. Example:

```
by.css('selector-name')
```

Specs and Po's

- It is common practice to divide each of protractor specs into “specs” themselves and “po”s (Page objects).
- While the “spec” part contains the test code, the “po” part — exports various selectors and methods specific to the page (which will be used by the “spec” part).
- The Page Objects allow us to create helper functions that will simplify our test code and save us having to repeat the same code multiple times in the one test. We can create a function in the Page Object file and reuse it as often as we need in the spec.ts file.

Protractor Example

Example “e2e” directory for an angular project with “login”, “protected” and “public” components . Best practice is create a directory structure that matches the angular code.

```
└─ e2e
  └─ login
    TS login.e2e-spec.ts
    TS login.po.ts
  └─ protected
    TS protected.e2e-spec.ts
    TS protected.po.ts
  └─ public
    TS public.e2e-spec.ts
    TS public.po.ts
```

Protractor Example

An easiest scenario-- “po” (page object) for “public” page

```
1  import { browser, by, element } from 'protractor';
2
3  export class PublicPage {
4      navigateTo() {
5          return browser.get('/'); // we can navigate to '/' for get public page since this is the default
6      }
7
8      getPageTitleText() {
9          return element(by.css('app-root h1')).getText();
10     }
11
12     logOut() {
13         return element(by.css('a[href="/login"]')).click();
14     }
15 }
```

Protractor Example

The “spec” part using “navigateTo” method defined in the “po” part

```
1  import { PublicPage } from './public.po';
2
3  describe('protractor-tutorial - Public page', () => {
4    let page: PublicPage;
5
6    beforeEach(() => {
7      page = new PublicPage();
8    });
9
10   it('when user browses to our app he should see the default “public” screen', () => {
11     page.navigateTo();
12     expect(page.getPageTitleText()).toEqual('Public');
13   });
14 });
```

Protractor Example

Run: **ng e2e**

If everything is intact you should see the following console output:

```
protractor-tutorial - Public page  
  ✓ when user browses to our app he should see the default "public" screen
```

Protractor Example

“po” for
“login” page

```
1  import { browser, by, element } from 'protractor';
2
3  export class LoginPage {
4      private credentials = {
5          username: 'test',
6          password: 'test'
7      };
8
9      navigateTo() {
10         return browser.get('/login');
11     }
12
13     fillCredentials(credentials: any = this.credentials) {
14         element(by.css('[name="username"]')).sendKeys(credentials.username);
15         element(by.css('[name="password"]')).sendKeys(credentials.password);
16         element(by.css('.btn-primary')).click();
17     }
18
19     getPageTitleText() {
20         return element(by.css('app-root h2')).getText();
21     }
22
23     getErrorMessage() {
24         return element(by.css('.alert-danger')).getText();
25     }
26 }
```

Protractor Example

The “spec” part for
“login” page

```
login.e2e-spec.ts x
1 import { LoginPage } from '../login.po';
2 import { PublicPage } from '../public/public.po';
3 describe('protractor-tutorial - Login page', () => {
4   let page: LoginPage;
5   let publicPage: PublicPage;
6
7   const wrongCredentias = {
8     username: 'wrongname',
9     password: 'wrongpasswd'
10  };
11
12  beforeEach(() => {
13    page = new LoginPage();
14    publicPage = new PublicPage();
15  });
16
17  it('when user trying to login with wrong credentials he should stay on “login” page', () => {
18    page.navigateTo();
19    page.fillCredentials(wrongCredentias);
20    expect(page.getPageTitleText()).toEqual('Login');
21    expect(page.getErrorMessage()).toEqual('Username or password is incorrect');
22  });
23
24  it('when login is successful – he should redirect to default “public” page', () => {
25    page.navigateTo();
26    page.fillCredentials();
27    expect(publicPage.getPageTitleText()).toEqual('Public');
28  });
29 });
30
```

Protractor Example

The “po” for “protected” page

```
ts protected.po.ts x
1  import { browser, by, element } from 'protractor';
2
3  export class ProtectedPage {
4      navigateTo() {
5          return browser.get('/protected');
6      }
7
8      getPageTitleText() {
9          return element(by.css('app-root h1')).getText();
10     }
11 }
12
```

Protractor Example

The “spec”
for “protected”
page

TS protected.e2e-spec.ts X

```
1 import { ProtectedPage } from './protected.po';
2 import { PublicPage } from '../public/public.po';
3 import { LoginPage } from '../login/login.po';
4 describe('protractor-tutorial - Protected page', () => {
5     let page: ProtectedPage;
6     let publicPage: PublicPage;
7     let loginPage: LoginPage;
8     beforeEach(() => {
9         page = new ProtectedPage();
10        publicPage = new PublicPage();
11        loginPage = new LoginPage();
12    });
13
14    it('when not authenticated user tries to access “protected” page he should redirect to “login” page and see error not
15        publicPage.navigateTo();
16        publicPage.logOut(); // must be logged out before trying access "protected" page
17        page.navigateTo();
18        expect(loginPage.getPageTitleText()).toEqual('Login');
19    });
20
21    it('not authenticated user tries to access “protected” page and after being redirected to “login” screen, after succe
22    login he should redirect to “protected”', () => {
23        publicPage.navigateTo();
24        publicPage.logOut(); // must be logged out before trying access "protected" page
25        page.navigateTo();
26        loginPage.fillCredentials();
27        expect(page.getPageTitleText()).toEqual('Protected');
28    });
29 });
```

Protractor Example

Run
again: **ng
e2e**

If everything
is intact you
should see
the console
output:

```
DevTools listening on ws://127.0.0.1:12587/devtools/browser/21d80879-f61f-48d9-90ae-fe6b0d4fa971
 Jasmine started

protractor-tutorial - Login page
  ✓ when user trying to login with wrong credentials he should stay on "login" page
  ✓ when login is successful – he should redirect to default "public" page

protractor-tutorial - Protected page
  ✓ when not authenticated user tries to access "protected" page he should redirect to "login" page and see error notification
  ✓ not authenticated user tries to access "protected" page and after being redirected to "login" screen, after successful login he should redirect to "protected"

protractor-tutorial - Public page
  ✓ when user browses to our app he should see the default "public" screen

Executed 5 of 5 specs SUCCESS in 9 secs.
[09:48:50] I/launcher - 0 instance(s) of WebDriver still running
[09:48:50] I/launcher - chrome #01 passed
```

Reading materials

1. Jasmine's [GitHub page](https://jasmine.github.io/api/edge/matchers.html) <https://jasmine.github.io/api/edge/matchers.html>
2. Protractor API <http://www.protractortest.org/#/api>
3. **Building Your First Tests For Angular5 With Protractor**

<https://blog.cloudboost.io/building-your-first-tests-for-angular5-with-protractor-a48dfc225a75>

4. Getting Started With End-to-End Testing in Angular Using Protractor

<https://code.tutsplus.com/tutorials/getting-started-with-end-to-end-testing-in-angular-using-protractor--cms-29318>